

Материалы кафедры ММП факультета ВМК МГУ. Введение в глубокое обучение.

Занятие 09. Цикл обучения

Занятие провел: Оганов Александр
(@wel mud)

Материалы составили: Оганов Александр
(@wel mud), Находнов Максим
(nakhodnov17@gmail.com)

Москва, Весенний семестр 2026

Источники:

- [Сборка нейронной сети PyTorch](#)
- [Обучение классификатора для CIFAR-10](#)
- [Reproducibility PyTorch](#)
- [Подробное описание получения state-of-the-art \(SOTA\) результатов.](#)
- Ноутбук во многом опирается на [этот конспект](#)

В этом ноутбуке вы познакомитесь с тем, как писать цикл обучения нейронных сетей с помощью библиотеки PyTorch. В процессе обучения вы будете наблюдать изменение метрик, значений функций потерь и других показателей, для отслеживания которых их нужно правильно логировать. В ноутбуке вы обучите свёрточную нейронную сеть (CNN) для классификации датасета [Imagenette](#) и сравните с transfer learning.

Цикл обучения

Цикл обучения содержит в себе все необходимые взаимодействия между данными, нейронной сетью и оптимизатором для обучения. Кроме того, он должен быть написан достаточно высокоуровнево, чтобы незначительные изменения каждого компонента не влияли на код обучения.

Идейно цикл обучения выглядит следующим образом:

1. Получение батча данных из датасета;
2. Вычисление нейронной сети на батче данных;
3. Вычисление функции ошибок;
4. Рассчёт градиентов с помощью обратного распространения (backpropagation);
5. Шаг оптимизации;
6. Переход на пункт 1.

По [ссылке](#) можно посмотреть логи wandb для всех экспериментов. В ноутбуке в качестве логера используется comet_ml. Логи для него доступны [по ссылке](#).

Важно: желательно использовать GPU, в ином случае модель будет учиться долго.

Для начала нам необходимо загрузить библиотеки. Также, мы установим режим вывода графиков в векторные форматы.

```
In [1]: import os
import random

import numpy as np
import pandas as pd
from PIL import Image
from sklearn.preprocessing import OrdinalEncoder
from torch.utils.data import DataLoader, Dataset
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

import matplotlib.pyplot as plt
import matplotlib_inline

%matplotlib inline
matplotlib_inline.backend_inline.set_matplotlib_formats('pdf', 'svg')
```

Работа с данными

Загрузка данных

Самым известным датасетом для классификации изображений является [ImageNet](#). В

нем содержится 1000 классов и более 14 миллионов изображений. Обучение моделей классификации даже с помощью современных видеокарт на нём является трудной задачей.

Мы рассмотрим датасет Imagenette, который является подмножеством из 10 самых легко классифицируемых классов датасета Imagenet. Кроме того, Imagenette удобно реализован в `torchvision.datasets.Imagenette`, что делает его удобным датасетом для сравнения разных методов. Мы не будем пользоваться `torchvision`, так как мы ставим перед собой цель научиться обучать модели от начала до конца. Мы скачаем Imagenette с официального [GitHub](#).

[PyTorch страница Imagenette](#).

```
In [2]: DATA_DIR = "./imagenette2-160/"

if not os.path.exists(DATA_DIR):
    !wget https://s3.amazonaws.com/fast-ai-imageclas/imagenette2-160.tgz
    !tar -xf ./imagenette2-160.tgz

os.listdir(DATA_DIR)
```

```
--2026-02-25 23:47:16-- https://s3.amazonaws.com/fast-ai-imageclas/imagenette2-160.tgz
Распознаётся s3.amazonaws.com (s3.amazonaws.com)... 16.182.64.192, 54.231.200.48, 16.15.203.127, ...
Подключение к s3.amazonaws.com (s3.amazonaws.com)|16.182.64.192|:443... соединение установлено.
HTTP-запрос отправлен. Ожидание ответа... 200 OK
Длина: 99003388 (94M) [application/x-tar]
Сохранение в: «imagenette2-160.tgz»

imagenette2-160.tgz 100%[=====>] 94,42M 5,87MB/s за 19s

2026-02-25 23:47:37 (4,85 MB/s) - «imagenette2-160.tgz» сохранён [99003388/99003388]
```

```
Out[2]: ['train', 'noisy_imagenette.csv', '.DS_Store', 'val']
```

Обработка данных

Внимательно посмотрим на данные, для этого прочитаем их в `pd.DataFrame`.

```
In [3]: data_df_path = os.path.join(DATA_DIR, 'noisy_imagenette.csv')

data_df = pd.read_csv(data_df_path)
data_df.head()
```

```
Out[3]:
```

	path	noisy_labels_0	noisy_labels_1	noisy_l
0	train/n02979186/n02979186_9036.JPEG	n02979186	n02979186	n02
1	train/n02979186/n02979186_11957.JPEG	n02979186	n02979186	n02
2	train/n02979186/n02979186_9715.JPEG	n02979186	n02979186	n02
3	train/n02979186/n02979186_21736.JPEG	n02979186	n02979186	n02
4	train/n02979186/ILSVRC2012_val_00046953.JPEG	n02979186	n02979186	n02

Вопрос: Почему для одного объекта есть несколько noisy_labels_*?

```
In [4]: tain_val_count = np.unique(data_df.is_valid, return_counts=True)[1].tolist()

print(f"В обучающей выборке {tain_val_count[0]} объектов")
print(f"В валидационной выборке {tain_val_count[1]} объектов")
```

В обучающей выборке 9469 объектов
В валидационной выборке 3925 объектов

Путь до изображения лежит в `data_df['path']`, метка класса хранится в `data_df_path['noisy_labels_0']`. Нам будет удобнее работать с числовым представлением класса, поэтому переведем каждую метку класса в число с помощью `OrdinalEncoder`.

```
In [5]: enc = OrdinalEncoder()
labels = data_df['noisy_labels_0'].values
labels_enc = enc.fit_transform(labels[:, None]).astype('int').ravel()

names, counts = np.unique(labels_enc, return_counts=True)
print("Информация о таргетах")
print("Присутствуют классы:", names.tolist())
print("Количество объектов каждого класса:", counts.tolist())

data_df['target'] = labels_enc
data_df.head()
```

Информация о таргетах
Присутствуют классы: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Количество объектов каждого класса: [1350, 1350, 1350, 1244, 1350, 1350, 1350, 1350, 1350, 1350]

```
Out[5]:
```

	path	noisy_labels_0	noisy_labels_1	noisy_l
0	train/n02979186/n02979186_9036.JPEG	n02979186	n02979186	n02
1	train/n02979186/n02979186_11957.JPEG	n02979186	n02979186	n02
2	train/n02979186/n02979186_9715.JPEG	n02979186	n02979186	n02
3	train/n02979186/n02979186_21736.JPEG	n02979186	n02979186	n02
4	train/n02979186/ILSVRC2012_val_00046953.JPEG	n02979186	n02979186	n02

Работа с данными в PyTorch

Идейно цикл обучения выглядит следующим образом:

1. Получение батча данных из датасета;
2. Вычисление нейронной сети на батче данных;
3. Вычисление функции ошибок;
4. Рассчёт градиентов с помощью обратного распространения (backpropagation);
5. Шаг оптимизации;
6. Переход на пункт 1.

Многие из конструкций выше уже реализованы в PyTorch. Нам не нужно писать их самостоятельно, так как внутри они реализуют весь нужный функционал наиболее эффективно.

Для работы с датасетом нам следует обернуть данные в

`torch.utils.data.Dataset`, так как это удобная обертка для данных, которая унифицирует к ним доступ. Использование `torch.utils.data.Dataset` позволяет удобно и эффективно итерироваться и получать доступ к данным. Класс наследник `torch.utils.data.Dataset` должен переопределять магические методы `__len__` и `__getitem__`.

Важно: одна из проблем нейронных сетей заключается в переобучении. Для борьбы с ним обычно используют аугментации и трансформации данных. Обычно все аугментации и трансформации применяются к изображениям при обращении к элементам датасета. Такой подход позволяет не хранить аугментированную выборку, что очень важно в случае непрерывных изменений (поворот, изменения контраста, яркости и тому подобное). Нейронная сеть, оптимизатор и другие абстракции внутри цикла обучения (кроме датасета) не должны быть зависимы от аугментаций данных.

```
In [6]: from torch.utils.data import Dataset, DataLoader
```

```
In [7]: class ImageDataset(Dataset):

    def __init__(self, data_df, is_train, transform):
        super().__init__()
        mask = data_df['is_valid'] != is_train

        self.img_paths = data_df[mask].path.values
        self.targets = data_df[mask].target.values

        self.transform = transform

    def __getitem__(self, idx):
        path = self.img_paths[idx]
        img_path = os.path.join(DATA_DIR, path)

        # YOUR CODE HERE
        img = Image.open(img_path).convert('RGB')
        img = self.transform(img)
```

```

        target = self.targets[idx]

        return img, target

    def __len__(self):
        return len(self.targets)

```

Для получения батча мы воспользуемся `torch.utils.data.DataLoader`, который позволяет быстро получить батч данных. Реализация `Dataloader` – достаточно не тривиальный функционал. Подробно обо всех параметрах можно почитать в документации, но мы обсудим основные.

```

DataLoader(
    dataset: torch.utils.data.dataset.Dataset[+T_co],
    batch_size: Optional[int] = 1,
    shuffle: Optional[bool] = None,
    sampler: Union[torch.utils.data.sampler.Sampler, Iterable,
NoneType] = None,
    batch_sampler: Union[torch.utils.data.sampler.Sampler[List],
Iterable[List], NoneType] = None,
    num_workers: int = 0,
    collate_fn: Optional[Callable[[List[~T]], Any]] = None,
    pin_memory: bool = False,
    drop_last: bool = False,
    timeout: float = 0,
    worker_init_fn: Optional[Callable[[int], NoneType]] = None,
    multiprocessing_context=None,
    generator=None,
    *,
    prefetch_factor: Optional[int] = None,
    persistent_workers: bool = False,
    pin_memory_device: str = '',
)

```

- `dataset` – объект класса `torch.utils.data.Dataset`
- `batch_size` – размер батча
- `shuffle` – надо ли перемешивать выборку после каждой эпохи
- `sampler` – чаще всего вы будете его оставлять равным `None`, но через этот параметр можно определить более сложное поведение выбора объектов в батче
- `num_workers` – часто получение объекта – сложная задача (например, обращение к диску для считывания изображения), поэтому бывает удобно делать это в несколько потоков. Параметр отвечает за количество созданных потоков. **Google Colab** не поддерживает `num_workers > 0`
- `collate_fn` – функция, с помощью которой собирается батч, чаще всего `None`. Подробнее вы узнаете в домашних заданиях и на следующих занятиях.

- `drop_last` – если у нас выборка из 10 изображений, а батч равен 9, то мы не хотим на второй итерации считать градиенты по одному элементу. При `drop_last` последний батч меньшего размера будет пропускаться.

Остальные параметры мы либо разберем ниже, либо на других парах, либо оставим для любознательного читателя.

Вопрос: Как отличаются параметры Dataloader при обучении и валидации?

Рассмотрим пример работы с Dataloader с батчем 9. В качестве датасета мы возьмем тензор размера 10. После каждой эпохи мы будем перемешивать данные и не учитывать последний батч, если он меньше остальных.

```
In [8]: D = DataLoader(
    torch.arange(10),
    batch_size=9,
    shuffle=True,
    drop_last=True
)

for epoch in range(5):
    for batch in D:
        print(epoch, batch)
```

```
0 tensor([2, 3, 5, 7, 8, 9, 6, 4, 0])
1 tensor([1, 6, 3, 2, 5, 4, 0, 8, 7])
2 tensor([9, 1, 3, 8, 0, 5, 6, 7, 2])
3 tensor([2, 7, 6, 9, 5, 8, 0, 3, 4])
4 tensor([6, 1, 9, 5, 3, 7, 0, 2, 8])
```

Аугментация и преобразования

В библиотеке `torchvision` есть отдельный модуль, в котором содержатся различные аугментации. Рассмотреть все аугментации мы не сможем, но достаточно подробный разбор можно найти [на официальном сайте](#).

Все классические аугментации хранятся в модулях `torchvision.transforms` (подходит для задачи классификации) и `torchvision.transforms.v2`, который содержит больше аугментаций для сегментации и детекции объектов.

[Полный список аугментаций](#).

[Визуализации некоторых аугментаций](#).

Рассмотрим основной набор преобразований:

1. Тип 1

А. Вырезать центр изображения с фиксированным размером

2. Тип 2

- A. Вырезать случайный участок фиксированного размера из изображения.
- B. Развернуть изображение горизонтально с вероятностью 0.5.
- C. Перевести в черно-белую палитру с вероятностью 0.5.
- D. Применить [эластичное преобразование](#).

3. Тип 3

- A. Вырезать случайный участок фиксированного размера из изображения.
- B. Развернуть изображение горизонтально с вероятностью 0.5.
- C. Применить случайное аффинное преобразование.
- D. Применить случайное удаление участка изображения.
- E. Инвертировать изображение с вероятностью 0.5.

4. Тип 4

- A. Применить случайное [стандартное преобразование](#).
- B. Изменить размер изображения до заданного.

Ниже посмотрим на результаты каждого преобразования на одном наборе изображений.

```
In [9]: import torchvision.transforms as transforms
```

```
In [10]: transforms_dict = {}

IMG_SIZE = 120

transforms_dict["Тип 1"] = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.CenterCrop((IMG_SIZE, IMG_SIZE)),
    ]
)

transforms_dict["Тип 2"] = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.RandomCrop((IMG_SIZE, IMG_SIZE)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomGrayscale(p=0.5),
        transforms.ElasticTransform()
    ]
)

transforms_dict["Тип 3"] = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.RandomCrop((IMG_SIZE, IMG_SIZE)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomAffine(40),
        transforms.RandomErasing(p=0.5),
        transforms.RandomInvert(p=0.5)
    ]
)
```



```
)

transforms_dict["Тип 4"] = transforms.Compose(
    [
        transforms.TrivialAugmentWide(),
        transforms.ToTensor(),
        transforms.Resize((IMG_SIZE, IMG_SIZE))
    ]
)
```

```
In [11]: fig, ax = plt.subplots(2, 2, figsize=(8, 8))

for idx, (aug_name, transform) in enumerate(transforms_dict.items()):

    vis_dataset = ImageDataset(data_df=data_df, is_train=True, transform=tra
    vis_dataloader = DataLoader(vis_dataset, batch_size=16, shuffle=False, r
    batch = next(iter(vis_dataloader))

    img_grid = torchvision.utils.make_grid(batch[0], nrow=4)

    ax[idx // 2, idx % 2].imshow(img_grid.permute(1, 2, 0).detach().cpu().nu
    ax[idx // 2, idx % 2].axis('off')
    ax[idx // 2, idx % 2].set_title(aug_name, fontsize=18)

plt.tight_layout()
plt.show()
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

Вопрос: Какой будет вывод, если запустить ячейку выше повторно? Почему это плохо?

Подготовка данных к обучению

Выберем минимальный набор аугментаций для нашей задачи. Наша цель – не выбить выше качество, а написать хороший цикл обучения. Хорошим тоном является нормализация данных, ведь так нейронная сеть будет учиться быстрее. Мы будем нормализовать изображения с помощью статистик, которые посчитали по всему ImageNet .

`IMG_MEAN[i]` содержит среднее значение пикселей для `i` канала.

`IMG_STD[i]` содержит стандартное отклонение пикселей для `i` канала.

```
In [12]: IMG_MEAN = np.array([0.485, 0.456, 0.406])
        IMG_STD  = np.array([0.229, 0.224, 0.225])

        transform_train = transforms.Compose(
            [
                transforms.ToTensor(),
                transforms.Normalize(mean=IMG_MEAN, std=IMG_STD),
                transforms.RandomCrop((160, 160)),
                transforms.RandomHorizontalFlip(p=0.5),
                transforms.RandomGrayscale(p=0.1),
            ]
        )

        transform_val = transforms.Compose(
            [
                transforms.ToTensor(),
                transforms.Normalize(mean=IMG_MEAN, std=IMG_STD),
                transforms.CenterCrop((160, 160))
            ]
        )
```

Вопрос: Почему преобразования для `train` и `val` разные?

```
In [13]: train_dataset = ImageDataset(data_df, is_train=True, transform=transform_train)
        valid_dataset = ImageDataset(data_df, is_train=False, transform=transform_val)

        len(train_dataset), len(valid_dataset)
```

Out[13]: (9469, 3925)

```
In [14]: vis_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, num_workers=4)

        batch = next(iter(vis_dataloader))

        print(f"Размерность батча: {batch[0].shape}")
        print(f"Классы в батче: {batch[1]}")
```

Размерность батча: torch.Size([16, 3, 160, 160])

Классы в батче: tensor([3, 6, 0, 3, 7, 0, 9, 1, 9, 5, 0, 6, 7, 4, 8, 1])

Обязательно посмотрим на данные и как они выглядят после аугментаций. Иногда при **неправильных** аугментациях можно испортить изображение.

```
In [15]: img_grid = torchvision.utils.make_grid(batch[0], nrow=4)

        plt.figure(figsize=(8, 8))
        plt.imshow(img_grid.permute(1, 2, 0).detach().cpu().numpy() * IMG_STD + IMG_MEAN)
        plt.axis('off')

        plt.tight_layout()
        plt.show()
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

Мы удостоверились, что данные корректны, поэтому перейдем к написанию цикла обучения.

Цикл обучения

Выше мы описали все необходимое для правильной и аккуратной работы с данными, а именно создали `Dataset` с изображениями и аугментациями, который используется для `Dataloader`. Благодаря этому мы можем не думать о работе с данными, а эффективно итерироваться по батчам.

Нам осталось определить:

- `нейронную сеть` – то, что будет обучаться;
- `оптимизатор` – то, как будет обучаться;

- функцию потерь – то, по чему будем обучаться.

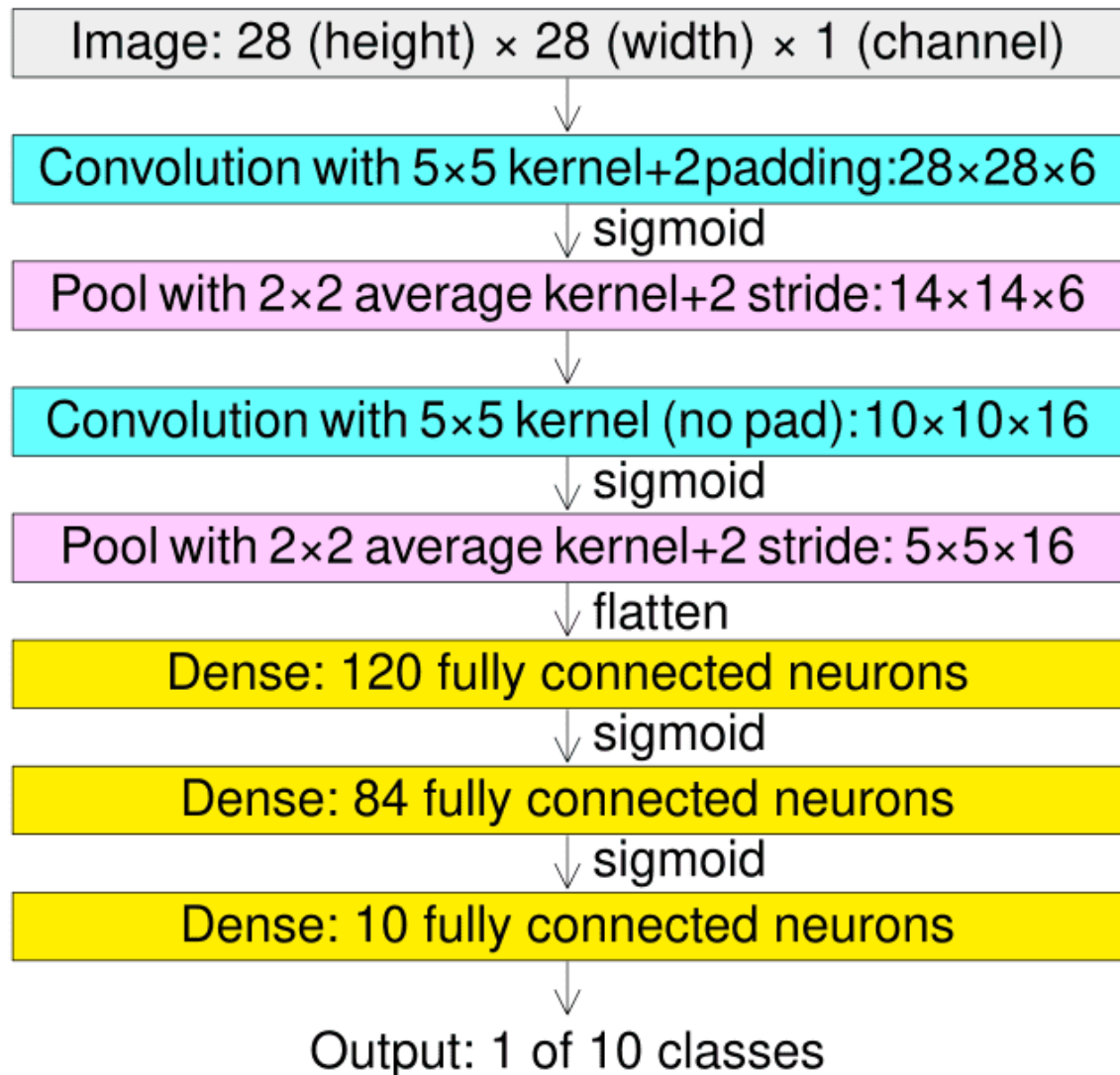
После этого мы сможем написать **цикл обучения**, в котором соединим все вместе.

LeNet для классификации

[Le et al., 1998](#) | примерно 80 тысяч цитат

Мы будем использовать свёрточные архитектуры, в частности реализуем что-то похожее на стандартную сеть **LeNet**. Мы могли бы начать с более сложных сетей, но у нас нет необходимости получить высокое качество.

LeNet



Источник

```
In [ ]: import torch
```

```
import torch.nn as nn
import torch.optim as optim
```

```
In [ ]: if torch.cuda.is_available():
        device = 'cuda:0'
    else:
        print("Мы рекомендуем использовать GPU.")
        device = 'cpu'
    device
```

```
Out[ ]: 'cuda:0'
```

Создадим класс нейронной сети LeNet, который является наследником `torch.nn.Module`. Важно не забыть про `super().__init__()`.

```
In [ ]: class LeNet(torch.nn.Module):
        def __init__(self, n_classes):
            super().__init__()

            self.features = torch.nn.Sequential(
                torch.nn.Conv2d(in_channels=3, out_channels=6, kernel_size=(5, 5),
                                torch.nn.MaxPool2d(kernel_size=(2, 2), stride=(2, 2)),
                                torch.nn.ReLU(),

                torch.nn.Conv2d(in_channels=6, out_channels=6, kernel_size=(5, 5),
                                torch.nn.MaxPool2d(kernel_size=(2, 2), stride=(2, 2)),
                                torch.nn.ReLU(),

                torch.nn.Conv2d(in_channels=6, out_channels=16, kernel_size=(5, 5),
                                torch.nn.MaxPool2d(kernel_size=(2, 2), stride=(2, 2)),
                                torch.nn.ReLU(),
            )

            self.dense = torch.nn.Sequential(
                torch.nn.Flatten(),
                torch.nn.Linear(16 * 16 * 16, 120),
                torch.nn.ReLU(),
                torch.nn.Linear(120, 84),
                torch.nn.ReLU(),
                torch.nn.Linear(84, n_classes),
            )

        def forward(self, x):
            x = self.features(x)
            x = self.dense(x)

            return x
```

Проверим, что мы не ошиблись в размерностях. Самой первой проверкой является создание прямого прохода для случайного батча.

```
In [ ]: net = LeNet(n_classes=10)

        batch = next(iter(vis_data_loader))
```

```
out = net(batch[0])

print(f"Размерность выхода: {out.shape}")
```

Размерность выхода: torch.Size([16, 10])

Посмотрим на число параметров, чтобы примерно оценить сложность модели.

```
In [ ]: def print_params_count(model):

    total_params = sum(p.numel() for p in model.parameters())
    total_params_grad = sum(p.numel() for p in model.parameters() if p.requires_grad_())

    model_name = model.__class__.__name__
    print(f"Информация о числе параметров модели: {model_name}")
    print(f"Всего параметров: \t\t {total_params}")
    print(f"Всего обучаемых параметров: \t {total_params_grad}")
    print()
```

```
In [ ]: print_params_count(net)
```

Информация о числе параметров модели: LeNet
Всего параметров: 506432
Всего обучаемых параметров: 506432

В качестве оптимизатора возьмем `Adam`, который является стандартом области.

```
In [ ]: optimizer = optim.Adam(net.parameters(), lr=1e-4)
```

Так как мы решаем задачу классификации, то будем использовать `torch.nn.CrossEntropyLoss()`.

```
In [ ]: loss_fn = torch.nn.CrossEntropyLoss()
```

```
In [ ]: train_dataloader = DataLoader(
    train_dataset,
    batch_size = 256,
    shuffle     = True,
    drop_last   = True,
    num_workers = 0,
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size = 4096,
    shuffle     = False,
    num_workers = 0,
)
```

Наконец-то мы можем написать **наш первый цикл обучения (train loop)**! Напомним его структуру:

1. Получение батча данных из датасета;

2. Вычисление нейронной сети на батче данных;
3. Вычисление функции ошибок;
4. Рассчёт градиентов с помощью обратного распространения (backpropagation);
5. Шаг оптимизации;
6. Переход на пункт 1.

```
In [ ]: def train_base(epoch_num, net, optimizer, loss_fn, train_dataloader, device)

    ### YOUR CODE HERE

    global_step = 0
    net = net.to(device)
    net.train()

    for _ in tqdm(range(epoch_num)):

        for X_batch, y_true in train_dataloader:
            optimizer.zero_grad()

            X_batch = X_batch.to(device)
            y_true = y_true.to(device)

            out = net(X_batch)

            loss = loss_fn(out, y_true)
            loss.backward()

            optimizer.step()

            y_pred = torch.argmax(out, 1)
            accuracy = torch.sum(y_pred == y_true) / y_pred.shape[0]
            print(f"{global_step}, loss = {loss.item():0.3f}, \t accuracy =

            global_step += 1
```

Обучим 1 эпоху, чтобы проверить корректность работы. Если мы нигде не ошиблись, то функция потерь (loss) должна стать меньше, а точность (accuracy) выше.

```
In [ ]: epoch_num = 1
train_base(
    epoch_num=epoch_num,
    net=net,
    optimizer=optimizer,
    loss_fn=loss_fn,
    train_dataloader=train_dataloader,
    device=device
)
```

0% | | 0/1 [00:00<?, ?it/s]

```
0, loss = 2.319, accuracy = 0.0664
1, loss = 2.306, accuracy = 0.0938
2, loss = 2.299, accuracy = 0.0781
3, loss = 2.297, accuracy = 0.1133
4, loss = 2.296, accuracy = 0.1445
5, loss = 2.292, accuracy = 0.1133
6, loss = 2.294, accuracy = 0.1250
7, loss = 2.291, accuracy = 0.0977
8, loss = 2.289, accuracy = 0.0820
9, loss = 2.292, accuracy = 0.0859
10, loss = 2.288, accuracy = 0.0977
11, loss = 2.283, accuracy = 0.1211
12, loss = 2.272, accuracy = 0.1758
13, loss = 2.282, accuracy = 0.1211
14, loss = 2.278, accuracy = 0.1406
15, loss = 2.267, accuracy = 0.1797
16, loss = 2.272, accuracy = 0.1445
17, loss = 2.272, accuracy = 0.1719
18, loss = 2.277, accuracy = 0.1875
19, loss = 2.253, accuracy = 0.2148
20, loss = 2.247, accuracy = 0.2109
21, loss = 2.260, accuracy = 0.1797
22, loss = 2.257, accuracy = 0.1758
23, loss = 2.271, accuracy = 0.1172
24, loss = 2.263, accuracy = 0.1797
25, loss = 2.236, accuracy = 0.1992
26, loss = 2.250, accuracy = 0.1914
27, loss = 2.251, accuracy = 0.1758
28, loss = 2.250, accuracy = 0.1523
29, loss = 2.240, accuracy = 0.1836
30, loss = 2.223, accuracy = 0.2148
31, loss = 2.231, accuracy = 0.1836
32, loss = 2.204, accuracy = 0.2148
33, loss = 2.194, accuracy = 0.2461
34, loss = 2.202, accuracy = 0.2656
100% |██████████| 1/1 [00:18<00:00, 18.33s/it]
35, loss = 2.193, accuracy = 0.2617
```

Судя по выводу, модель учится: точность растет, функция потерь уменьшается.

Обратите внимание, что у нас 10 классов, а точность предсказания $\sim 16\%$, что выше случайного угадывания (10%).

Такой вывод не позволит понять, что происходит с моделью при тысячах итераций, так как можно будет легко запутаться.

Вопрос: Как улучшить вывод модели?

Для ответа на этот вопрос, мы воспользуемся библиотеками для логирования (`tensorboard`, `wandb`, `comet_ml`, `trackio`, `ml_flow`). В этом ноутбуке мы рассмотрим `comet_ml` как наиболее комфортный логер в использовании.

Логирование

Основы логирования

Мы будем использовать библиотеку `comet_ml`. Большинство логеров имеют схожий синтаксис и возможности. Мы рекомендуем попробовать `wandb`, код ниже для него закомментирован. `wandb` более известный и удобный логер, но у вас может возникнуть необходимость в сторонних сервисах для комфортного использования.

Библиотеки для логирования созданы с целью простого отслеживания метрик и результатов экспериментов. Опишем основные функции от очевидных до более неожиданных:

- Логирование чисел
- Логирование рисунков
- Логирование на удаленный сервер
- Логирование архитектуры
- Сохранение файлов
- Сохранение состояний git
- Визуализация модели
- Построение графов гиперпараметров
- Автоматический поиск гиперпараметров и их визуализация

[Подробная документация comet_ml](#).

[Подробная документация wandb](#) может не работать без сторонних сервисов.

```
In [ ]: !pip install comet_ml -q
```



The image shows two progress bars for the pip installation. The first bar is for the main package and shows 780.9/780.9 kB at 19.2 MB/s with an eta of 0:0. The second bar is for a dependency and shows 1.3/1.3 MB at 73.3 MB/s with an eta of 0:00:00.

```
In [ ]: import wandb
import comet_ml
```

При первом запуске вам потребуется указать API ключ, его можно получить, если зарегистрироваться на официальном сайте. Для логирования кода в `wandb` достаточно указать `save_code=True`.

[По ссылке](#) можно посмотреть логи `wandb` для всех экспериментов.

[По ссылке](#) можно посмотреть логи `comet_ml` для всех экспериментов.

Для начала нужно инициализировать запуск. Ниже мы создали запуск, указали его имя (если не указать явно, то будет случайное имя), сохранили гиперпараметры, сохранили код. Это базовый минимум, который нужно сделать в хорошем проекте при инициализации логера.

```
In [16]: # Использовать если на GoogleColab, иначе подставить свой api
```

```
# from google.colab import userdata
```

```
In [ ]: project_name = 'testing_2025'
run_name = 'test'
config = {"conf": "testing"}

experiment = comet_ml.start(
    api_key=userdata.get('COMET_API_KEY'),
    project_name=project_name
)

experiment.set_name(run_name)
experiment.log_parameters(config)

for path in os.listdir('.'):
    if path.endswith('.ipynb'):
        experiment.log_code(file_name=path)

# wandb version

# run = wandb.init(
#     project=project_name,
#     name=run_name,
#     config=config,
#     save_code=True
# )

# wandb.run.log_code(
#     "./",
#     include_fn=lambda path: path.endswith(".ipynb")
# )
```

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET WARNING: As you are running in a Jupyter environment, you will need to call `experiment.end()` when finished to ensure all metrics and code are logged before exiting.

COMET INFO: Experiment is live on comet.com <https://www.comet.com/enteralt/testing-2025/c0c662da0cbb4bc597e82211a95b263c>

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent directory. Set `COMET\_GIT\_DIRECTORY` if your Git Repository is elsewhere.

Логирование чисел.

```
In [ ]: experiment.log_metrics({"train/loss": 0, "eval/loss": 0}, step=0)

# wandb version
# wandb.log({"train/loss": 0, "eval/loss": 0}, step=0)
```

Важно: Параметр `step` должен расти монотонно!

Логирование изображений. Важно не забывать про нормализацию изображений при логировании.

```
In [ ]: experiment.log_image(image_data=transforms.functional.to_pil_image(img_grid))

# wandb version
# wandb.log({"media/img": wandb.Image(img_grid)}, step=0)
```

```
Out[ ]: {'web': 'https://www.comet.com/api/image/download?imageId=82761eb5ab27468f8e2a9f060bcbcbba0&experimentKey=c0c662da0cbb4bc597e82211a95b263c',
  'api': 'https://www.comet.com/api/rest/v1/image/get-image?imageId=82761eb5ab27468f8e2a9f060bcbcbba0&experimentKey=c0c662da0cbb4bc597e82211a95b263c',
  'imageId': '82761eb5ab27468f8e2a9f060bcbcbba0'}
```

Логирование гистограмм

```
In [ ]: experiment.log_histogram_3d(values=torch.arange(100) ** 2, name='media/hist')

# wandb version
# wandb.log({"media/hist": wandb.Histogram(torch.arange(100) ** 2)}, step=0)
```

```
Out[ ]: {'web': 'https://www.comet.com/api/asset/download?assetId=582d4135275d4ca1a9e3796f09ebf78c&experimentKey=c0c662da0cbb4bc597e82211a95b263c',
  'api': 'https://www.comet.com/api/rest/v2/experiment/asset/get-asset?assetId=582d4135275d4ca1a9e3796f09ebf78c&experimentKey=c0c662da0cbb4bc597e82211a95b263c',
  'assetId': '582d4135275d4ca1a9e3796f09ebf78c'}
```

Логирование графиков.

```
In [ ]: fig = plt.figure(figsize=(6,6))
plt.plot(np.arange(100) ** 2)
plt.title(r"$y = x^2$")

experiment.log_figure.figure=fig, figure_name='media/plots', step=0)

# wandb version
# wandb.log({"media/plots": wandb.Image(fig)}, step=0)

plt.close('all') # мы не хотим отображать графики в ноутбук
```

Посмотрим на запуск, в нем должны отобразиться логированные метрики. Иногда метрики логируются не сразу, а после заполнения некоего буфера, так как синхронизация с веб сайтом – дорогая операция.

```
In [ ]: experiment.log_metrics({"train/loss": 2, "eval/loss": 1}, step=1)
experiment.log_metrics({"train/loss": 5, "eval/loss": -1}, step=2)

fig = plt.figure(figsize=(6,6))
plt.plot(np.arange(100) ** 3)
plt.title(r"$y = x^3$")

experiment.log_figure.figure=fig, figure_name='media/plots', step=3)
experiment.log_histogram_3d(values=torch.arange(100) ** 3, name='media/hist')
```

```
# Следующие шаги для визуализаций wandb
# wandb.log({"train/loss": 2, "eval/loss": 1}, step=1)
# wandb.log({"media/hist": wandb.Histogram(torch.arange(100) ** 3)}, step=1)

# fig = plt.figure(figsize=(6,6))
# plt.plot(np.arange(100) ** 3)
# plt.title(r"$y = x^3$")

# wandb.log({"media/plots": wandb.Image(fig)}, step=1)

# # Сравнить в чем отличие
# wandb.log({"media/img": wandb.Image(img_grid.permute(1, 2, 0).numpy() / 4)
# wandb.log({"media/img": wandb.Image(img_grid / 4)}, step=2)

plt.close('all') # мы не хотим отображать графики в ноутбуке
```

Обязательно нужно завершить запуск.

```
In [ ]: experiment.end()

# wandb version
# wandb.finish()
```

```

COMET INFO: -----
COMET INFO: Comet.ml Experiment Summary
COMET INFO: -----
COMET INFO: Data:
COMET INFO:   display_summary_level : 1
COMET INFO:   name                  : test
COMET INFO:   url                   : https://www.comet.com/enteralt/testing-2025/c0c662da0cbb4bc597e82211a95b263c
COMET INFO: Metrics [count] (min, max):
COMET INFO:   eval/loss [3]  : (-1, 1)
COMET INFO:   train/loss [3] : (0, 5)
COMET INFO: Others:
COMET INFO:   Name          : test
COMET INFO:   notebook_url  : https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych
COMET INFO: Parameters:
COMET INFO:   conf : testing
COMET INFO: Uploads:
COMET INFO:   environment details : 1
COMET INFO:   figures              : 2
COMET INFO:   filename            : 1
COMET INFO:   histogram3d         : 2
COMET INFO:   images              : 1
COMET INFO:   installed packages  : 1
COMET INFO:   notebook            : 2
COMET INFO:   os packages         : 1
COMET INFO:   source_code         : 1
COMET INFO:
COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

```

У `wandb` есть неочевидные преимущества:

1. при отображении графиков на сайте, можно отобразить все точки, а в `comet_ml` не более 1000 (по умолчанию 50);
2. `wandb` лучше работает с изображениями и нормализует их автоматически;
3. для логирования `wandb` достаточно обращаться к глобальной переменной `wandb`. В случае `comet_ml` нужно использовать локальную переменную `experiment`. Обойти это можно с помощью команды `comet_ml.get_running_experiment()`.
4. `wandb` не требует уточнения типа данных при логировании, всё логируется через `wandb.log`.

Добавление логирования в цикл обучения.

```

In [ ]: def train_logger(epoch_num, net, optimizer, loss_fn, train_data_loader, device):
        # ДОБАВИЛИ

```

```

experiment = comet_ml.start(
    api_key=userdata.get('COMET_API_KEY'),
    project_name="cnn_mmp_2025"
)
experiment.set_name(run_name)
for path in os.listdir('./'):
    if path.endswith('.ipynb'):
        experiment.log_code(file_name=path)

# wandb.init(
#     project="cnn_mmp_2025",
#     name=run_name,
#     save_code=True
# )
# wandb.run.log_code(
#     "./",
#     include_fn=lambda path: path.endswith(".ipynb")
# )

global_step = 0
net = net.to(device)

for _ in tqdm(range(epoch_num)):
    net.train()

    for X_batch, y_true in train_dataloader:
        optimizer.zero_grad()

        X_batch = X_batch.to(device)
        y_true = y_true.to(device)

        out = net(X_batch)

        loss = loss_fn(out, y_true)
        loss.backward()

        optimizer.step()

        y_pred = torch.argmax(out, 1)
        accuracy = torch.sum(y_pred == y_true) / y_pred.shape[0]

        # ДОБАВИЛИ
        experiment.log_metrics({"train/loss": loss.item(), "train/accuracy": accuracy})
        # wandb.log({"train/loss": loss.item(), "train/accuracy": accuracy})

        global_step += 1

# ДОБАВИЛИ
experiment.end()
# wandb.finish()

```

Вопрос: Можно ли сразу запустить обучение? Например, сразу запустить код ниже.

In []: # epoch_num = 1


```
# train_logger(epoch_num, net, optimizer, loss_fn, train_dataloader, valid_da
```

```
In [ ]: net = LeNet(10)
optimizer = optim.Adam(net.parameters(), lr=1e-4)
```

```
In [ ]: epoch_num = 10

train_logger(
    epoch_num=epoch_num,
    net=net,
    optimizer=optimizer,
    loss_fn=loss_fn,
    train_dataloader=train_dataloader,
    device=device,
    run_name="LeNet_base"
)
```

```

COMET WARNING: To get all data logged automatically, import comet_ml before the
following modules: sklearn, torch.
COMET WARNING: As you are running in a Jupyter environment, you will need to
call `experiment.end()` when finished to ensure all metrics and code are log
ged before exiting.
COMET INFO: Experiment is live on comet.com https://www.comet.com/enteralt/cn
n-mm-p-2025/58463f5ddd7c4b71aa86791d6a6248ef

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent di
rectory. Set `COMET_GIT_DIRECTORY` if your Git Repository is elsewhere.
100%|██████████| 10/10 [02:45<00:00, 16.58s/it]
COMET INFO: -----
COMET INFO: Comet.ml Experiment Summary
COMET INFO: -----
COMET INFO: Data:
COMET INFO:   display_summary_level : 1
COMET INFO:   name                  : LeNet_base
COMET INFO:   url                   : https://www.comet.com/enteralt/cnn-mm
p-2025/58463f5ddd7c4b71aa86791d6a6248ef
COMET INFO: Metrics [count] (min, max):
COMET INFO:   train/accuracy [360] : (0.078125, 0.515625)
COMET INFO:   train/loss [360]    : (1.5339734554290771, 2.30494546890258
8)
COMET INFO: Others:
COMET INFO:   Name           : LeNet_base
COMET INFO:   notebook_url  : https://colab.research.google.com/notebook#fil
eId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych
COMET INFO: Uploads:
COMET INFO:   environment details : 1
COMET INFO:   filename            : 1
COMET INFO:   installed packages  : 1
COMET INFO:   notebook            : 2
COMET INFO:   os packages         : 1
COMET INFO:   source_code         : 1
COMET INFO:
COMET WARNING: To get all data logged automatically, import comet_ml before the
following modules: sklearn, torch.
COMET INFO: Please wait for assets to finish uploading (timeout is 10800 seco
nds)
COMET INFO: All assets have been sent, waiting for delivery confirmation

```

Вопрос: Чего не хватает?

Валидация

Обычно, модель выбирают по метрикам на валидационной выборке. Кроме того, валидация – хороший способ заметить переобучение. Без валидации мы ничего не знаем о качестве модели.

```

In [ ]: @torch.no_grad()
def evaluate(net, valid_dataloader, loss_fn, device):

```

```

#### YOUR CODE HERE

net.eval()
loss, accuracy = 0, 0
count = 0
for X_batch, y_true in valid_dataloader:
    X_batch = X_batch.to(device)
    y_true = y_true.to(device)

    out = net(X_batch)
    y_pred = torch.argmax(out, 1)

    bs = out.shape[0]
    loss += loss_fn(out, y_true).item() * bs
    accuracy += torch.sum(y_pred == y_true).item()
    count += bs

return loss / count, accuracy / count

```

Вопрос: Что делает декоратор `torch.no_grad`?

```

In [ ]: def train_logger_eval(epoch_num, net, optimizer, loss_fn, train_dataloader, v

    experiment = comet_ml.start(
        api_key=userdata.get('COMET_API_KEY'),
        project_name="cnn_mmp_2025"
    )
    experiment.set_name(run_name)
    for path in os.listdir('./'):
        if path.endswith('.ipynb'):
            experiment.log_code(file_name=path)

    # wandb.init(
    #     project="cnn_mmp_2025",
    #     name=run_name,
    #     save_code=True
    # )
    # wandb.run.log_code(
    #     "./",
    #     include_fn=lambda path: path.endswith(".ipynb")
    # )

    global_step = 0
    net = net.to(device)

    for _ in tqdm(range(epoch_num)):
        net.train()

        for X_batch, y_true in train_dataloader:
            optimizer.zero_grad()

            X_batch = X_batch.to(device)
            y_true = y_true.to(device)

```

```

        out = net(X_batch)

        loss = loss_fn(out, y_true)
        loss.backward()

        optimizer.step()

        y_pred = torch.argmax(out, 1)
        accuracy = torch.sum(y_pred == y_true) / y_pred.shape[0]

        experiment.log_metrics({"train/loss": loss.item(), "train/accuracy": accuracy})
        # wandb.log({"train/loss": loss.item(), "train/accuracy": accuracy})

        global_step += 1

    # ДОБАВИЛИ
    loss, accuracy = evaluate(
        net=net,
        valid_dataloader=valid_dataloader,
        loss_fn=loss_fn,
        device=device
    )
    experiment.log_metrics({"eval/loss": loss, "eval/accuracy": accuracy})
    # wandb.log({"eval/loss": loss, "eval/accuracy": accuracy}, step=global_step)

experiment.end()
# wandb.finish()

```

```

In [ ]: net = LeNet(10)
        optimizer = optim.Adam(net.parameters(), lr=1e-4)

```

```

In [ ]: epoch_num = 10
        train_logger_eval(
            epoch_num      = epoch_num,
            net             = net,
            optimizer       = optimizer,
            loss_fn         = loss_fn,
            train_dataloader = train_dataloader,
            valid_dataloader = valid_dataloader,
            device          = device,
            run_name        = "LeNet_eval"
        )

```

```

COMET WARNING: To get all data logged automatically, import comet_ml before the
following modules: sklearn, torch.
COMET WARNING: As you are running in a Jupyter environment, you will need to
call `experiment.end()` when finished to ensure all metrics and code are log
ged before exiting.
COMET INFO: Experiment is live on comet.com https://www.comet.com/enteralt/cnn-mm
p-2025/9a12d227ab2d4066bf117d8320cc8d8b

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent di
rectory. Set `COMET_GIT_DIRECTORY` if your Git Repository is elsewhere.
100%|██████████| 10/10 [03:25<00:00, 20.54s/it]
COMET INFO: -----
-----
COMET INFO: Comet.ml Experiment Summary
COMET INFO: -----
-----
COMET INFO: Data:
COMET INFO:   display_summary_level : 1
COMET INFO:   name                  : LeNet_eval
COMET INFO:   url                   : https://www.comet.com/enteralt/cnn-mm
p-2025/9a12d227ab2d4066bf117d8320cc8d8b
COMET INFO: Metrics [count] (min, max):
COMET INFO:   eval/accuracy [10] : (0.2624203821656051, 0.426496815286624
2)
COMET INFO:   eval/loss [10]   : (1.6807193756103516, 2.217403888702392
6)
COMET INFO:   train/accuracy [360] : (0.0546875, 0.48046875)
COMET INFO:   train/loss [360]   : (1.6077303886413574, 2.311273336410522
5)
COMET INFO: Others:
COMET INFO:   Name                : LeNet_eval
COMET INFO:   notebook_url       : https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych
COMET INFO: Uploads:
COMET INFO:   environment details : 1
COMET INFO:   filename            : 1
COMET INFO:   installed packages  : 1
COMET INFO:   notebook            : 2
COMET INFO:   os packages         : 1
COMET INFO:   source_code         : 1
COMET INFO:
COMET WARNING: To get all data logged automatically, import comet_ml before the
following modules: sklearn, torch.
COMET INFO: Please wait for assets to finish uploading (timeout is 10800 seco
nds)
COMET INFO: All assets have been sent, waiting for delivery confirmation

```

Вопрос: Чего не хватает?

Кризис воспроизводимости

Выше мы могли заметить, что обучение сильно меняется от запуска к запуску, что создает следующие проблемы:

- Возможно, кто-то другой не сможет воспроизвести ваши результаты
- Возможно, **вы** никогда не сможете воспроизвести свои же результаты

Получить крутую модель, но без возможности повторения, крайне плохо для науки, индустрии, домашней работы и просто грустно. Для воспроизводимых результатов недостаточно просто зафиксировать `seed`. Например, ниже продемонстрирована зависимость генератора случайных чисел от устройства.

Важно: Невоспроизводимая работа **не** имеет ценности!

```
In [ ]: torch.manual_seed(42)
        randn1 = torch.randn(10)

        torch.manual_seed(42)
        randn2 = torch.randn(10)

        randn1, randn2
```

```
Out[ ]: (tensor([ 0.3367,  0.1288,  0.2345,  0.2303, -1.1229, -0.1863,  2.2082, -0.
        6380,
                0.4617,  0.2674]),
        tensor([ 0.3367,  0.1288,  0.2345,  0.2303, -1.1229, -0.1863,  2.2082, -0.
        6380,
                0.4617,  0.2674]))
```

```
In [ ]: torch.manual_seed(42)
        randn1 = torch.randn(10)

        torch.manual_seed(42)
        randn2 = torch.randn(10, device='cuda')

        randn1, randn2
```

```
Out[ ]: (tensor([ 0.3367,  0.1288,  0.2345,  0.2303, -1.1229, -0.1863,  2.2082, -0.
        6380,
                0.4617,  0.2674]),
        tensor([ 0.1940,  2.1614, -0.1721,  0.8491, -1.9244,  0.6530, -0.6494, -0.
        8175,
                0.5280, -1.2753], device='cuda:0'))
```

Для начала постараемся сделать обучение независимым от очередности запуска, воспользуемся советами из официальной статьи [Reproducibility PyTorch](#). Нам необходимо:

- Зафиксировать сид CPU;
- Зафиксировать сид GPU;
- Зафиксировать сид `numpy`;
- Зафиксировать сид стандартного `random`;

В целом, этого уже будет достаточно и намного лучше, чем в большинстве современных работ. Малой кровью можно зафиксировать DataLoader, такое тоже будет полезно. Кроме того, существуют рандомизированные алгоритмы и алгоритмы, зависящие от модели технического оборудования. Эту случайность намного сложнее контролировать, но можно с помощью

`torch.use_deterministic_algorithms(True)` – так гарантируется точная воспроизводимость, но код замедлится, скорее всего.

На практике можно почти все случайности зафиксировать и гарантировать детерминизм. Откуда возникает вопрос: "Будет ли хорошей модель, которая перестает учиться, если чуть иначе зашафлить датасет или чуть иначе умножать матрицы?"

```
In [ ]: def set_global_seed(seed: int) -> None:
        """Set global seed for reproducibility.
        :param int seed: Seed to be set
        """

        torch.manual_seed(seed)
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        np.random.seed(seed)
        random.seed(seed)
        # если нужно гарантировать 1000% воспроизводимость
        # torch.use_deterministic_algorithms(False)

        # Для DataLoader
        g = torch.Generator()
        g.manual_seed(seed)

        return g

        # Для каждого worker в DataLoader
        def seed_worker(worker_id):
            worker_seed = torch.initial_seed() % 2**32
            np.random.seed(worker_seed)
            random.seed(worker_seed)
```

Вопрос: Где ошибка в коде ниже?

```
In [ ]: # net = LeNet(10)
        # optimizer = optim.Adam(net.parameters(), lr=1e-4)

        # g = set_global_seed(42)

        # train_dataloader = DataLoader(
        #     train_dataset,
        #     batch_size      = 256,
        #     shuffle          = True,
        #     drop_last        = True,
        #     num_workers       = 0,
        #     worker_init_fn   = seed_worker,
```

```

#     generator      = g
# )

# valid_dataloader = DataLoader(
#     valid_dataset,
#     batch_size     = 4096,
#     shuffle        = False,
#     num_workers    = 4
# )

# epoch_num = 10
# train_logger_eval(
#     epoch_num      = epoch_num,
#     net            = net,
#     optimizer      = optimizer,
#     loss_fn        = loss_fn,
#     train_dataloader = train_dataloader,
#     valid_dataloader = valid_dataloader,
#     device         = device,
#     run_name       = "LeNet_seed"
# )

```

```

In [ ]: g = set_global_seed(42)

train_dataloader = DataLoader(
    train_dataset,
    batch_size     = 256,
    shuffle        = True,
    drop_last      = True,
    num_workers    = 0,
    worker_init_fn = seed_worker,
    generator      = g
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size     = 4096,
    shuffle        = False,
    num_workers    = 0
)

net = LeNet(10)
optimizer = optim.Adam(net.parameters(), lr=1e-4)

```

```

In [ ]: epoch_num = 10
train_logger_eval(
    epoch_num      = epoch_num,
    net            = net,
    optimizer      = optimizer,
    loss_fn        = loss_fn,
    train_dataloader = train_dataloader,
    valid_dataloader = valid_dataloader,
    device         = device,
    run_name       = "LeNet_seed"
)

```



```

COMET WARNING: To get all data logged automatically, import comet_ml before the
following modules: sklearn, torch.
COMET WARNING: As you are running in a Jupyter environment, you will need to
call `experiment.end()` when finished to ensure all metrics and code are log
ged before exiting.
COMET INFO: Couldn't find a Git repository in '/content' nor in any parent di
rectory. Set `COMET_GIT_DIRECTORY` if your Git Repository is elsewhere.
COMET INFO: Experiment is live on comet.com https://www.comet.com/enteralt/cn
n-mm-p-2025/67142e586eee4baa8864c8304498fa80

100%|██████████| 10/10 [03:25<00:00, 20.57s/it]
COMET INFO: -----
-----
COMET INFO: Comet.ml Experiment Summary
COMET INFO: -----
-----
COMET INFO: Data:
COMET INFO:   display_summary_level : 1
COMET INFO:   name                  : LeNet_seed
COMET INFO:   url                   : https://www.comet.com/enteralt/cnn-mm
p-2025/67142e586eee4baa8864c8304498fa80
COMET INFO: Metrics [count] (min, max):
COMET INFO:   eval/accuracy [10] : (0.14522292993630573, 0.43974522292993
63)
COMET INFO:   eval/loss [10]    : (1.645544409751892, 2.253673791885376)
COMET INFO:   train/accuracy [360] : (0.0703125, 0.5078125)
COMET INFO:   train/loss [360]   : (1.5611698627471924, 2.31047630310058
6)
COMET INFO: Others:
COMET INFO:   Name                : LeNet_seed
COMET INFO:   notebook_url       : https://colab.research.google.com/notebook#fil
eId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych
COMET INFO: Uploads:
COMET INFO:   environment details : 1
COMET INFO:   filename            : 1
COMET INFO:   installed packages  : 1
COMET INFO:   notebook            : 2
COMET INFO:   os packages         : 1
COMET INFO:   source_code         : 1
COMET INFO:
COMET WARNING: To get all data logged automatically, import comet_ml before the
following modules: sklearn, torch.
COMET INFO: Please wait for assets to finish uploading (timeout is 10800 seco
nds)
COMET INFO: All assets have been sent, waiting for delivery confirmation

```

Вопрос: Чего не хватает?

Дополнительные логи

Очень часто на практике обучение в какой-то момент просто ломается. Причины бывают разные и чем сложнее ваша модель и задача, тем более разные причины. Возможно, где-то численная нестабильность, возможно, засэмплировался странный батч. Для этого бывает полезно логировать **веса** и **градиенты** параметров.

Логирование градиентов также позволит отслеживать затухание или взрывы, что часто бывает полезно.

```
In [ ]: @torch.no_grad()
def log_gradients(model, global_step, log_norm=True, log_hist=True):
    exp = comet_ml.get_running_experiment()

    for tag, value in model.named_parameters():
        g = value.grad
        if g is None:
            continue

        if log_hist:
            experiment.log_histogram_3d(values=g.cpu(), name=f'grad/{tag}',
                                       # wandb.log({f"grad/{tag}": wandb.Histogram(g.cpu())}, global_st

        if log_norm:
            experiment.log_metrics({f'grad_norm/{tag}': torch.norm(g.cpu())}
                                   # wandb.log({f"grad_norm/{tag}": torch.norm(g.cpu())}, global_st

@torch.no_grad()
def log_weights(model, global_step, log_norm=True, log_hist=True):
    exp = comet_ml.get_running_experiment()

    for tag, value in model.named_parameters():
        g = value.grad
        if g is None:
            continue

        if log_hist:
            experiment.log_histogram_3d(values=value.cpu(), name=f'weight/{t
                                       # wandb.log({f"weight/{tag}": wandb.Histogram(value.cpu())}, glo

        if log_norm:
            experiment.log_metrics({f'weight_norm/{tag}': torch.norm(value.c
                                   # wandb.log({f"weight_norm/{tag}": torch.norm(value.cpu())}, glo
```

```
In [ ]: def train_progress(config, net, optimizer, loss_fn, train_dataloader, valid_

    experiment = comet_ml.start(
        api_key=userdata.get('COMET_API_KEY'),
        project_name="cnn_mmp_2025"
    )
    experiment.set_name(config['name'])
    for path in os.listdir('./'):
        if path.endswith('.ipynb'):
            experiment.log_code(file_name=path)

    experiment.log_parameters(config) # ДОБАВИЛИ

    # wandb.init(
    #     project="cnn_mmp_2025",
    #     name=config['name'],
    #     save_code=True
    #     config=config # ДОБАВИЛИ
```

```

# )
# wandb.run.log_code(
#     "./",
#     include_fn=lambda path: path.endswith(".ipynb")
# )

global_step = 0
net = net.to(device)

epoch_num = config['epoch_num']
for _ in tqdm(range(epoch_num)):
    net.train()

    for X_batch, y_true in train_dataloader:
        optimizer.zero_grad()

        X_batch = X_batch.to(device)
        y_true = y_true.to(device)

        out = net(X_batch)

        loss = loss_fn(out, y_true)
        loss.backward()

        optimizer.step()

        # ДОБАВИЛИ
        log_gradients(net, global_step)
        log_weights(net, global_step)

        y_pred = torch.argmax(out, 1)
        accuracy = torch.sum(y_pred == y_true) / y_pred.shape[0]

        experiment.log_metrics({"train/loss": loss.item(), "train/accuracy": accuracy})
        # wandb.log({"train/loss": loss.item(), "train/accuracy": accuracy})

        global_step += 1

    loss, accuracy = evaluate(
        net = net,
        valid_dataloader = valid_dataloader,
        loss_fn = loss_fn,
        device = device
    )
    experiment.log_metrics({"eval/loss": loss, "eval/accuracy": accuracy})
    # wandb.log({"eval/loss": loss, "eval/accuracy": accuracy}, step=global_step)

experiment.end()
# wandb.finish()

```

Выше мы добавили новые логи, но также хотелось бы запоминать гиперпараметры. Например, мы увидели, что какой-то запуск выбивает очень хорошие метрики. Как вы будете воспроизводить этот запуск? Скорее всего, на практике вы забудете с какими параметрами вы запускали ваш код. Поэтому лучше сохранять все гиперпараметры

внутри одного конфига. Так вы избавитесь и от магических констант и повысите воспроизводимость.

```
In [ ]: # ДОБАВИЛИ
config = {
    'seed'           : 42,
    'lr'             : 1e-4,
    'epoch_num'      : 10,
    'batch_size'     : 256,
    'val_batch_size' : 4096,
    'name'           : 'LeNet_progress'
}

g = set_global_seed(config['seed'])

train_dataloader = DataLoader(
    train_dataset,
    batch_size     = config['batch_size'],
    shuffle        = True,
    drop_last      = True,
    num_workers    = 0,
    worker_init_fn = seed_worker,
    generator      = g
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size     = config['val_batch_size'],
    shuffle        = False,
    num_workers    = 0
)

net = LeNet(10)
optimizer = optim.Adam(net.parameters(), lr=config['lr'])
```

```
In [ ]: train_progress(
    config          = config,
    net             = net,
    optimizer       = optimizer,
    loss_fn         = loss_fn,
    train_dataloader = train_dataloader,
    valid_dataloader = valid_dataloader,
    device          = device
)
```

```

COMET WARNING: To get all data logged automatically, import comet_ml before the
following modules: sklearn, torch.
COMET WARNING: As you are running in a Jupyter environment, you will need to
call `experiment.end()` when finished to ensure all metrics and code are log
ged before exiting.
COMET INFO: Couldn't find a Git repository in '/content' nor in any parent di
rectory. Set `COMET_GIT_DIRECTORY` if your Git Repository is elsewhere.
COMET INFO: Experiment is live on comet.com https://www.comet.com/enteralt/cn
n-mmmp-2025/dea8d40231bf4c3ab57a0432f50eedee

100%|██████████| 10/10 [03:50<00:00, 23.07s/it]
COMET INFO: -----
-----
COMET INFO: Comet.ml Experiment Summary
COMET INFO: -----
-----
COMET INFO: Data:
COMET INFO:   display_summary_level : 1
COMET INFO:   name                  : LeNet_progress
COMET INFO:   url                    : https://www.comet.com/enteralt/cnn-mm
p-2025/dea8d40231bf4c3ab57a0432f50eedee
COMET INFO: Metrics [count] (min, max):
COMET INFO:   eval/accuracy [10]      : (0.14598726114649682, 0.43617834394904
46)
COMET INFO:   eval/loss [10]         : (1.649621605873108, 2.2534019947052)
COMET INFO:   train/accuracy [360]   : (0.0703125, 0.4921875)
COMET INFO:   train/loss [360]      : (1.5620405673980713, 2.31047630310058
6)
COMET INFO: Others:
COMET INFO:   Name                  : LeNet_progress
COMET INFO:   notebook_url         : https://colab.research.google.com/notebook#fil
eId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych
COMET INFO: Parameters:
COMET INFO:   batch_size           : 256
COMET INFO:   epoch_num            : 10
COMET INFO:   lr                   : 0.0001
COMET INFO:   name                 : LeNet_progress
COMET INFO:   seed                 : 42
COMET INFO:   val_batch_size       : 4096
COMET INFO: Uploads:
COMET INFO:   environment details : 1
COMET INFO:   filename             : 1
COMET INFO:   installed packages  : 1
COMET INFO:   notebook            : 2
COMET INFO:   os packages         : 1
COMET INFO:   source_code          : 1
COMET INFO:
COMET WARNING: To get all data logged automatically, import comet_ml before the
following modules: sklearn, torch.

```

Вопрос: Чего не хватает?

Чекпоинты

Обучение модели занимает много времени и ресурсов, но обычно модели учат не

ради высокой метрики на валидации, а ради дальнейшего использования. Для того, чтобы использовать модель в будущем, нужно явно прописать сохранения. Сохранять можно либо каждые N итераций, либо по метрике на валидации. Файл, в котором содержатся веса модели и иная важная информация об обучении, называется **чекпойнтом** (точка сохранения).

- Обучить модель – хорошо.
- Обучить модель и получить воспроизводимые результаты – отлично
- Обучить модель и получить воспроизводимые результаты, и сохранить веса – **идеально**

```
In [ ]: def train_final(config, net, optimizer, loss_fn, train_dataloader, valid_data_loader):

    experiment = comet_ml.start(
        api_key=userdata.get('COMET_API_KEY'),
        project_name="cnn_mmp_2025"
    )
    experiment.set_name(config['name'])
    for path in os.listdir('.'):
        if path.endswith('.ipynb'):
            experiment.log_code(file_name=path)

    experiment.log_parameters(config)

    # wandb.init(
    #     project="cnn_mmp_2025",
    #     name=config['name'],
    #     save_code=True
    #     config=config
    # )
    # wandb.run.log_code(
    #     "./",
    #     include_fn=lambda path: path.endswith(".ipynb")
    # )

    global_step = 0
    net = net.to(device)

    # ДОБАВИЛИ
    best_accuracy = 0
    os.makedirs(config['checkpoint_dir'], exist_ok=True)

    epoch_num = config['epoch_num']
    for epoch in tqdm(range(epoch_num)):
        net.train()

        for X_batch, y_true in train_dataloader:
            optimizer.zero_grad()

            X_batch = X_batch.to(device)
            y_true = y_true.to(device)
```

```

        out = net(X_batch)

        loss = loss_fn(out, y_true)
        loss.backward()

        optimizer.step()

        log_gradients(net, global_step)
        log_weights(net, global_step)

        y_pred = torch.argmax(out, 1)
        accuracy = torch.sum(y_pred == y_true) / y_pred.shape[0]

        experiment.log_metrics({"train/loss": loss.item(), "train/accuracy": accuracy})
        # wandb.log({"train/loss": loss.item(), "train/accuracy": accuracy})

        global_step += 1

    loss, accuracy = evaluate(
        net = net,
        valid_dataloader = valid_dataloader,
        loss_fn = loss_fn,
        device = device
    )
    experiment.log_metrics({"eval/loss": loss, "eval/accuracy": accuracy})
    # wandb.log({"eval/loss": loss, "eval/accuracy": accuracy}, step=global_step)

    # ДОБАВИЛИ
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        # YOUR CODE HERE
        torch.save({
            'epoch': epoch,
            'iter': global_step,
            'model_state_dict': net.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'loss': loss,
            'metric': accuracy
        }, os.path.join(config['checkpoint_dir'], config['checkpoint_name'] + str(global_step) + '.pth'))

    experiment.end()
    # wandb.finish()

```

```

In [ ]: # ДОБАВИЛИ
config = {
    'seed' : 42,
    'lr' : 1e-4,
    'epoch_num' : 10,
    'batch_size' : 256,
    'val_batch_size' : 4096,
    'name' : 'LeNet_final',
    'checkpoint_dir' : './checkpoints',
    'checkpoint_name' : 'LeNet.pth'
}

g = set_global_seed(config['seed'])

```

```

g.manual_seed(0)

train_dataloader = DataLoader(
    train_dataset,
    batch_size = config['batch_size'],
    shuffle     = True,
    drop_last   = True,
    num_workers = 0,
    worker_init_fn = seed_worker,
    generator    = g
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size = config['val_batch_size'],
    shuffle     = False,
    num_workers = 0
)

net = LeNet(10)
optimizer = optim.Adam(net.parameters(), lr=config['lr'])

```

```

In [ ]: train_final(
    config          = config,
    net             = net,
    optimizer       = optimizer,
    loss_fn         = loss_fn,
    train_dataloader = train_dataloader,
    valid_dataloader = valid_dataloader,
    device          = device
)

```


COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET WARNING: As you are running in a Jupyter environment, you will need to call `experiment.end()` when finished to ensure all metrics and code are logged before exiting.

COMET INFO: Experiment is live on comet.com <https://www.comet.com/enteralt/cnn-mm-p-2025/c2a25c2a47324509897595df67fe969c>

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent directory. Set `COMET\_GIT\_DIRECTORY` if your Git Repository is elsewhere.

100%|██████████| 10/10 [03:48<00:00, 22.85s/it]

COMET INFO: -----

COMET INFO: Comet.ml Experiment Summary

COMET INFO: -----

COMET INFO: Data:

COMET INFO: display_summary_level : 1

COMET INFO: name : LeNet_final

COMET INFO: url : <https://www.comet.com/enteralt/cnn-mm-p-2025/c2a25c2a47324509897595df67fe969c>

COMET INFO: Metrics [count] (min, max):

COMET INFO: eval/accuracy [10] : (0.14955414012738855, 0.42929936305732486)

COMET INFO: eval/loss [10] : (1.671627402305603, 2.251912832260132)

COMET INFO: train/accuracy [360] : (0.0703125, 0.48828125)

COMET INFO: train/loss [360] : (1.6110979318618774, 2.310384750366211)

COMET INFO: Others:

COMET INFO: Name : LeNet_final

COMET INFO: notebook_url : <https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych>

COMET INFO: Parameters:

COMET INFO: batch_size : 256

COMET INFO: checkpoint_dir : ./checkpoints

COMET INFO: checkpoint_name : LeNet.pth

COMET INFO: epoch_num : 10

COMET INFO: lr : 0.0001

COMET INFO: name : LeNet_final

COMET INFO: seed : 42

COMET INFO: val_batch_size : 4096

COMET INFO: Uploads:

COMET INFO: environment details : 1

COMET INFO: filename : 1

COMET INFO: installed packages : 1

COMET INFO: notebook : 2

COMET INFO: os packages : 1

COMET INFO: source_code : 1

COMET INFO:

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET INFO: Please wait for assets to finish uploading (timeout is 10800 seconds)

COMET INFO: All assets have been sent, waiting for delivery confirmation

Философия и немного рефлексии

Вопрос: Чего не хватает?

Как вы уже поняли, отвечать на этот вопрос можно бесконечно, но давайте попробуем это сделать в последний раз за сегодня.

Цикл обучения отражает то, что автор считает важным. Например, очень часто блок `.zero_grad()`, `.backward()`, `.step()` помещают внутрь отдельной функции или используют готовые фреймворки:

- [PyTorch Lightning](#)
- [HuggingFace trainer](#)
- [PyTorch Ignit](#)

В них весь цикл обучения уже записан и вам достаточно просто передать функцию, которая по батчу считает ошибку.

Почему же для такого простого цикла обучения кто-то пишет отдельные библиотеки?

Выше мы рассмотрели самую простую задачу без разных инженерных особенностей, но на самом деле можно:

- Нормировать градиенты
- Использовать смешанную точность
- Аккумулировать градиенты
- Использовать распределенное обучение
- Использовать экономию памяти GPU
- ...

Все это мы будем постепенно разбирать в будущем, пока мы научились самым основам, которые позволят идти дальше.

Важно: Цикл обучения в исследовательских задачах – это то, что делаете вы для себя и самое главное, чтобы вам было удобно. Кто-то пишет свои логи, а может вы найдете что-то свое.

Выше мы использовали подход обучения **эпохами**, но иногда бывает полезно обучать модели **итерациями**. Например, у нас нет конкретного обучающего датасета или же датасет слишком большой. В целом, подход через итерации более универсальный и можно записать функцию `train` следующим образом:

```
global_step = 0
```

```
for _ in tqdm(range(epoch)):
    for batch in data_loader:
```

```

net.train()
optim.zero_grad()

loss = net(batch)
loss.backward()

optim.step()

if global_step % config.eval_freq == 0:
    eval_loss = eval()

if global_step % config.checkpoint_freq == 0:
    torch.save({'model': model.state_dict()},
'checkpoint.pth')

if global_step % config.whatever_freq == 0:
    ...

if global_step == max_iter:
    return

global_step += 1

```

Результаты и исследование

LeNet – очень простая архитектура. Если мы хотим получить хорошее качество, нужно использовать что-то посложнее (например, взять модель из **torchvision.models**), а также использовать аугментацию данных. Другой путь улучшения результата – использование предобученной сети и **transfer learning**.

Мы хотим повысить качество, но пока рассмотрели только самую базовую свёрточную архитектуру. Прежде чем повысить качество, давайте попробуем получить некоторый **бейзлайн** – то, с чем мы будем сравнивать результаты.

Сейчас у нас есть свёрточный **бейзлайн** – **LeNet**

Ниже мы получим полносвязный **бейзлайн** – **MLP**

Дальше мы будем обучать **ResNet18**, у которого 11 миллионов параметров, поэтому у **MLP** должно быть сравнимое количество параметров, иначе мы не сможем сделать по результатам правильные выводы.

MLP

```

In [ ]: class DeepNetwork(nn.Module):
        def __init__(self, input_dim, hidden_dims, num_classes):
            super().__init__()

            self.fc_in = nn.Sequential(
                nn.Flatten(1),

```

```

        nn.Linear(in_features=input_dim, out_features=hidden_dims[0])
        nn.ReLU(),
    )

    self.layers = nn.Sequential(
        *[
            nn.Sequential(nn.Linear(h_in, h_out), nn.ReLU())
            for h_in, h_out in zip(hidden_dims[:-1], hidden_dims[1:])
        ],
    )

    self.fc_out = nn.Linear(hidden_dims[-1], num_classes)

    def forward(self, x):
        x = self.fc_in(x)
        x = self.layers(x)

        return self.fc_out(x)

```

```

In [ ]: print_params_count(LeNet(10))
        print_params_count(DeepNetwork(input_dim=160 * 160 * 3, hidden_dims=[128, 256, 256, 128]))

```

Информация о числе параметров модели: LeNet

Всего параметров: 506432

Всего обучаемых параметров: 506432

Информация о числе параметров модели: DeepNetwork

Всего параметров: 9963530

Всего обучаемых параметров: 9963530

```

In [ ]: config = {
    'seed'          : 42,
    'lr'            : 1e-4,
    'epoch_num'     : 10,
    'batch_size'    : 256,
    'val_batch_size': 4096,
    'name'          : 'MLP',
    'checkpoint_dir' : './checkpoints',
    'checkpoint_name': 'MLP.pth',
    'input_dim'     : 160 * 160 * 3,          # ДОБАВИЛИ
    'hidden_dims'   : [128, 256, 256, 128], # ДОБАВИЛИ
}
config['name'] = f"{config['name']}_{config['hidden_dims']}"

g = set_global_seed(config['seed'])
g.manual_seed(0)

train_dataloader = DataLoader(
    train_dataset,
    batch_size    = config['batch_size'],
    shuffle       = True,
    drop_last     = True,
    num_workers   = 0,
    worker_init_fn = seed_worker,
    generator     = g
)

```

```

)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size = config['val_batch_size'],
    shuffle     = False,
    num_workers = 0
)

net = DeepNetwork(
    input_dim   = config['input_dim'],
    hidden_dims = config['hidden_dims'],
    num_classes = 10
)
optimizer = optim.Adam(net.parameters(), lr=config['lr'])

```

```

In [ ]: train_final(
    config           = config,
    net              = net,
    optimizer        = optimizer,
    loss_fn          = loss_fn,
    train_dataloader = train_dataloader,
    valid_dataloader = valid_dataloader,
    device           = device
)

```

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET WARNING: As you are running in a Jupyter environment, you will need to call `experiment.end()` when finished to ensure all metrics and code are logged before exiting.

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent directory. Set `COMET\_GIT\_DIRECTORY` if your Git Repository is elsewhere.

COMET INFO: Experiment is live on comet.com <https://www.comet.com/enteralt/cnn-mm-p-2025/22b2dbad156f4b34b168d335344d0fec>

100%|██████████| 10/10 [07:52<00:00, 47.27s/it]

COMET INFO: -----

COMET INFO: Comet.ml Experiment Summary

COMET INFO: -----

COMET INFO: Data:

COMET INFO: display_summary_level : 1

COMET INFO: name : MLP_[128, 256, 256, 128]

COMET INFO: url : <https://www.comet.com/enteralt/cnn-mm-p-2025/22b2dbad156f4b34b168d335344d0fec>

COMET INFO: Metrics [count] (min, max):

COMET INFO: eval/accuracy [10] : (0.3378343949044586, 0.4631847133757962)

COMET INFO: eval/loss [10] : (1.6021476984024048, 1.8934917449951172)

COMET INFO: train/accuracy [360] : (0.11328125, 0.57421875)

COMET INFO: train/loss [360] : (1.3606700897216797, 2.296802043914795)

COMET INFO: Others:

COMET INFO: Name : MLP_[128, 256, 256, 128]

COMET INFO: notebook_url : <https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych>

COMET INFO: Parameters:

COMET INFO: batch_size : 256

COMET INFO: checkpoint_dir : ./checkpoints

COMET INFO: checkpoint_name : MLP.pth

COMET INFO: epoch_num : 10

COMET INFO: hidden_dims : [128, 256, 256, 128]

COMET INFO: input_dim : 76800

COMET INFO: lr : 0.0001

COMET INFO: name : MLP_[128, 256, 256, 128]

COMET INFO: seed : 42

COMET INFO: val_batch_size : 4096

COMET INFO: Uploads:

COMET INFO: environment details : 1

COMET INFO: filename : 1

COMET INFO: installed packages : 1

COMET INFO: notebook : 2

COMET INFO: os packages : 1

COMET INFO: source_code : 1

COMET INFO:

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

Бейзлан: обучение resnet18 с нуля

Чаще всего вы будете дообучать модели, так как учить модель с нуля очень дорого, а воспроизвести SOTA результаты еще труднее. Ниже мы будем исследовать разные подходы к `transfer learning`, но нам нужно иметь какой-то стандарт, с которым мы будем сравниваться. Поэтому, помимо `transfer learning`, мы еще и обучим модель сами.

Все эксперименты будем проводить с теми же параметрами обучения, что и раньше, чтобы мы могли ответить на вопрос: **"Как именно влияет архитектура?"**.

```
In [ ]: from torchvision.models import resnet18, ResNet18_Weights
```

```
In [ ]: net = resnet18()  
net
```

```

Out[ ]: ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
    bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_runnin
    g_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil
    _mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=
        (1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_ru
        nning_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=
        (1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_ru
        nning_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=
        (1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_ru
        nning_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=
        (1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_ru
        nning_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=
        (1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_r
        unning_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=
        (1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_r
        unning_stats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_r
          unning_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=
        (1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_r
        unning_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=

```



```

(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
    )
    )
    (layer3): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=
(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
        (downsample): Sequential(
          (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=Fals
e)
          (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
      )
    )
    (layer4): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=
(1, 1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
        (downsample): Sequential(
          (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=Fals
e)
          (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)

```

```

        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_r
unning_stats=True)
    )
    )
    (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
    (fc): Linear(in_features=512, out_features=1000, bias=True)
)

```

Посмотрим на число параметров.

```

In [ ]: print_params_count(LeNet(10))
        print_params_count(net)

```

Информация о числе параметров модели: LeNet
 Всего параметров: 506432
 Всего обучаемых параметров: 506432

Информация о числе параметров модели: ResNet
 Всего параметров: 11689512
 Всего обучаемых параметров: 11689512

Напишем класс обертку, чтобы было удобно работать с нейросетью. Кроме того, добавим параметры `mean` и `std`, так мы можем контролировать, что подается нейросети на вход.

Вопрос: Зачем может потребоваться добавление `mean` и `std` ?

```

In [ ]: class ResNetWrapper(nn.Module):
        def __init__(self, resnet_model, mean=0., std=1.):
            super().__init__()

            self.resnet_block = resnet_model
            self.std = std
            self.mean = mean

        def forward(self, x):

            x = x * self.std + self.mean

            # Зачем может потребоваться интерполяция?
            # Почему в случае ResNet она не нужна?
            x = torch.nn.functional.interpolate(x, scale_factor=1)
            x = self.resnet_block(x)

            return x

```

Обучим модель "с нуля", то есть без предобученных весов и без замороженных параметров.

```

In [ ]: config = {
    'seed'           : 42,
    'lr'             : 1e-4,
    'epoch_num'      : 10,
    'batch_size'      : 256,
    'val_batch_size' : 256,
    'name'           : 'ResNet18_train',
    'checkpoint_dir'  : './checkpoints',
    'checkpoint_name' : 'ResNet.pth'
}

g = set_global_seed(config['seed'])
g.manual_seed(0)

train_dataloader = DataLoader(
    train_dataset,
    batch_size     = config['batch_size'],
    shuffle        = True,
    drop_last      = True,
    num_workers    = 0,
    worker_init_fn = seed_worker,
    generator      = g
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size     = config['val_batch_size'],
    shuffle        = False,
    num_workers    = 0
)

# Добавили
resnet_model = resnet18()
# Почему делаем так?
resnet_model.fc = torch.nn.Linear(512, 10)

net = ResNetWrapper(resnet_model)
optimizer = optim.Adam(net.parameters(), lr=config['lr'])

```

```

In [ ]: train_final(
    config           = config,
    net              = net,
    optimizer        = optimizer,
    loss_fn          = loss_fn,
    train_dataloader = train_dataloader,
    valid_dataloader = valid_dataloader,
    device           = device
)

```

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET WARNING: As you are running in a Jupyter environment, you will need to call `experiment.end()` when finished to ensure all metrics and code are logged before exiting.

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent directory. Set `COMET\_GIT\_DIRECTORY` if your Git Repository is elsewhere.

COMET INFO: Experiment is live on comet.com <https://www.comet.com/enteralt/cnn-mm-p-2025/a4f05a2cf11141e7a88016d53caeb701>

100%|██████████| 10/10 [11:35<00:00, 69.56s/it]

COMET INFO: -----

COMET INFO: Comet.ml Experiment Summary

COMET INFO: -----

COMET INFO: Data:

COMET INFO: display_summary_level : 1

COMET INFO: name : ResNet18_train

COMET INFO: url : <https://www.comet.com/enteralt/cnn-mm-p-2025/a4f05a2cf11141e7a88016d53caeb701>

COMET INFO: Metrics [count] (min, max):

COMET INFO: eval/accuracy [10] : (0.44101910828025476, 0.7286624203821656)

COMET INFO: eval/loss [10] : (0.8187334305161883, 1.6473517207734905)

COMET INFO: train/accuracy [360] : (0.07421875, 0.87109375)

COMET INFO: train/loss [360] : (0.4694545269012451, 2.478025436401367)

COMET INFO: Others:

COMET INFO: Name : ResNet18_train

COMET INFO: notebook_url : <https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych>

COMET INFO: Parameters:

COMET INFO: batch_size : 256

COMET INFO: checkpoint_dir : ./checkpoints

COMET INFO: checkpoint_name : ResNet.pth

COMET INFO: epoch_num : 10

COMET INFO: lr : 0.0001

COMET INFO: name : ResNet18_train

COMET INFO: seed : 42

COMET INFO: val_batch_size : 256

COMET INFO: Uploads:

COMET INFO: environment details : 1

COMET INFO: filename : 1

COMET INFO: installed packages : 1

COMET INFO: notebook : 2

COMET INFO: os packages : 1

COMET INFO: source_code : 1

COMET INFO:

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

Transfer learning: finetuning

Теперь воспользуемся предобученной версией, которую учили на ImageNet с [разными лайфками](#). Мы применим finetuning, то есть возьмем веса обученной модели в качестве инициализации и будем обучать все веса модели под нашу задачу. Существуют подходы с более аккуратной настройкой весов, например, с помощью заморозки **только некоторых** слоев.

```
In [ ]: config = {
    'seed'           : 42,
    'lr'             : 1e-4,
    'epoch_num'      : 10,
    'batch_size'     : 256,
    'val_batch_size' : 256,
    'name'           : 'ResNet18_train_finnetuning',
    'checkpoint_dir' : './checkpoints',
    'checkpoint_name': 'ResNet_finnetuning.pth'
}

g = set_global_seed(config['seed'])

train_dataloader = DataLoader(
    train_dataset,
    batch_size    = config['batch_size'],
    shuffle       = True,
    drop_last     = True,
    num_workers   = 0,
    worker_init_fn = seed_worker,
    generator     = g
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size    = config['val_batch_size'],
    shuffle       = False,
    num_workers   = 0
)

resnet_model = resnet18(
    weights=ResNet18_Weights.IMAGENET1K_V1 # Добавили
)
resnet_model.fc = torch.nn.Linear(512, 10)

net = ResNetWrapper(resnet_model)
optimizer = optim.Adam(net.parameters(), lr=config['lr'])
```

Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /
root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth

100%|██████████| 44.7M/44.7M [00:00<00:00, 192MB/s]

```
In [ ]: train_final(config, net, optimizer, loss_fn, train_dataloader, valid_dataloader)
```

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET WARNING: As you are running in a Jupyter environment, you will need to call `experiment.end()` when finished to ensure all metrics and code are logged before exiting.

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent directory. Set `COMET\_GIT\_DIRECTORY` if your Git Repository is elsewhere.

COMET INFO: Experiment is live on comet.com <https://www.comet.com/enteralt/cnn-mm-p-2025/4e84562fb45943babbd33ede7b785f9c>

100%|██████████| 10/10 [11:18<00:00, 67.84s/it]

COMET INFO: -----

COMET INFO: Comet.ml Experiment Summary

COMET INFO: -----

COMET INFO: Data:

COMET INFO: display_summary_level : 1

COMET INFO: name : ResNet18_train_finetuning

COMET INFO: url : <https://www.comet.com/enteralt/cnn-mm-p-2025/4e84562fb45943babbd33ede7b785f9c>

COMET INFO: Metrics [count] (min, max):

COMET INFO: eval/accuracy [10] : (0.9485350318471337, 0.9689171974522293)

COMET INFO: eval/loss [10] : (0.0980249999245261, 0.1775663245559498)

COMET INFO: train/accuracy [360] : (0.04296875, 1.0)

COMET INFO: train/loss [360] : (0.004306132439523935, 2.646655559539795)

COMET INFO: Others:

COMET INFO: Name : ResNet18_train_finetuning

COMET INFO: notebook_url : <https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych>

COMET INFO: Parameters:

COMET INFO: batch_size : 256

COMET INFO: checkpoint_dir : ./checkpoints

COMET INFO: checkpoint_name : ResNet_finetuning.pth

COMET INFO: epoch_num : 10

COMET INFO: lr : 0.0001

COMET INFO: name : ResNet18_train_finetuning

COMET INFO: seed : 42

COMET INFO: val_batch_size : 256

COMET INFO: Uploads:

COMET INFO: environment details : 1

COMET INFO: filename : 1

COMET INFO: installed packages : 1

COMET INFO: notebook : 2

COMET INFO: os packages : 1

COMET INFO: source_code : 1

COMET INFO:

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET INFO: Please wait for assets to finish uploading (timeout is 10800 seconds)

COMET INFO: Still uploading 1 file(s), remaining 751.90 KB/4.86 MB

Transfer learning: linear probing

При `linear probing` мы будем учить только линейный слой (голову нейронной сети).

```
In [ ]: config = {
    'seed'           : 42,
    'lr'             : 1e-4,
    'epoch_num'      : 10,
    'batch_size'     : 256,
    'val_batch_size' : 256,
    'name'           : 'ResNet18_lp',
    'checkpoint_dir' : './checkpoints',
    'checkpoint_name': 'ResNet_lp.pth'
}

g = set_global_seed(config['seed'])

train_dataloader = DataLoader(
    train_dataset,
    batch_size    = config['batch_size'],
    shuffle       = True,
    drop_last     = True,
    num_workers   = 0,
    worker_init_fn = seed_worker,
    generator     = g
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size    = config['val_batch_size'],
    shuffle       = False,
    num_workers   = 0
)

resnet_model = resnet18(
    weights=ResNet18_Weights.IMAGENET1K_V1
)

# Добавили
for parameter in resnet_model.parameters():
    parameter.requires_grad_(False)

# Почему тут?
resnet_model.fc = torch.nn.Linear(512, 10)

net = ResNetWrapper(resnet_model)
optimizer = optim.Adam(net.parameters(), lr=config['lr'])
```

```
In [ ]: print_params_count(net)
```

Информация о числе параметров модели: ResNetWrapper
Всего параметров: 11181642
Всего обучаемых параметров: 5130

```
In [ ]: train_final(config, net, optimizer, loss_fn, train_dataloader, valid_dataloader)
```

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET WARNING: As you are running in a Jupyter environment, you will need to call `experiment.end()` when finished to ensure all metrics and code are logged before exiting.

COMET INFO: Experiment is live on comet.com <https://www.comet.com/enteralt/cnn-mm-p-2025/e2b279651cd846dbb664e551d45dda13>

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent directory. Set `COMET\_GIT\_DIRECTORY` if your Git Repository is elsewhere.

100%|██████████| 10/10 [03:46<00:00, 22.64s/it]

COMET INFO: -----

COMET INFO: Comet.ml Experiment Summary

COMET INFO: -----

COMET INFO: Data:

COMET INFO: display_summary_level : 1

COMET INFO: name : ResNet18_lp

COMET INFO: url : <https://www.comet.com/enteralt/cnn-mm-p-2025/e2b279651cd846dbb664e551d45dda13>

COMET INFO: Metrics [count] (min, max):

COMET INFO: eval/accuracy [10] : (0.26904458598726116, 0.9276433121019109)

COMET INFO: eval/loss [10] : (0.4743537257583278, 2.078793535050313)

COMET INFO: train/accuracy [360] : (0.03125, 0.9609375)

COMET INFO: train/loss [360] : (0.46511897444725037, 2.6972274780273438)

COMET INFO: Others:

COMET INFO: Name : ResNet18_lp

COMET INFO: notebook_url : <https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych>

COMET INFO: Parameters:

COMET INFO: batch_size : 256

COMET INFO: checkpoint_dir : ./checkpoints

COMET INFO: checkpoint_name : ResNet_lp.pth

COMET INFO: epoch_num : 10

COMET INFO: lr : 0.0001

COMET INFO: name : ResNet18_lp

COMET INFO: seed : 42

COMET INFO: val_batch_size : 256

COMET INFO: Uploads:

COMET INFO: environment details : 1

COMET INFO: filename : 1

COMET INFO: installed packages : 1

COMET INFO: notebook : 2

COMET INFO: os packages : 1

COMET INFO: source_code : 1

COMET INFO:

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

Out-of domain

Мы хотим понять, как работают предобученные веса на данных вне обучаемого домена. Например, если мы учились на данных с `mean=0`, а в нашем датасете данные с `mean=1`. Например, мы учились на изображениях кошек, но хотим использовать веса для обучения на изображениях собак.

Менять датасет мы не будем, просто поменяем `mean` и `std` в `ResNetWrapper`.

Вопрос: Что будет лучше: linear probing или finetuning? Какая у вас гипотеза? Почему?

```
In [ ]: config = {
    'seed'           : 42,
    'lr'             : 1e-4,
    'epoch_num'      : 10,
    'batch_size'     : 256,
    'val_batch_size' : 256,
    'name'           : 'ResNet18_train_finetuning_ood',
    'checkpoint_dir' : './checkpoints',
    'checkpoint_name': 'ResNet_finetuning.pth'
}

g = set_global_seed(config['seed'])

train_dataloader = DataLoader(
    train_dataset,
    batch_size    = config['batch_size'],
    shuffle       = True,
    drop_last     = True,
    num_workers   = 0,
    worker_init_fn = seed_worker,
    generator     = g
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size    = config['val_batch_size'],
    shuffle       = False,
    num_workers   = 0
)

resnet_model = resnet18(
    weights=ResNet18_Weights.IMAGENET1K_V1
)
resnet_model.fc = torch.nn.Linear(512, 10)

net = ResNetWrapper(
    resnet_model,
    mean = 4,  # Добавили
    std  = 0.1 # Добавили
)
optimizer = optim.Adam(net.parameters(), lr=config['lr'])
```

```
In [ ]: train_final(config, net, optimizer, loss_fn, train_dataloader, valid_dataloa
```

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET WARNING: As you are running in a Jupyter environment, you will need to call `experiment.end()` when finished to ensure all metrics and code are logged before exiting.

COMET INFO: Experiment is live on comet.com <https://www.comet.com/enteralt/cnn-mm-p-2025/060118f63a774f8c96139047cd6d8eee>

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent directory. Set `COMET\_GIT\_DIRECTORY` if your Git Repository is elsewhere.

100%|██████████| 10/10 [11:17<00:00, 67.71s/it]

COMET INFO: -----

COMET INFO: Comet.ml Experiment Summary

COMET INFO: -----

COMET INFO: Data:

COMET INFO: display_summary_level : 1

COMET INFO: name : ResNet18_train_finetuning_ood

COMET INFO: url : <https://www.comet.com/enteralt/cnn-mm-p-2025/060118f63a774f8c96139047cd6d8eee>

COMET INFO: Metrics [count] (min, max):

COMET INFO: eval/accuracy [10] : (0.8440764331210191, 0.9454777070063695)

COMET INFO: eval/loss [10] : (0.17123333519431436, 0.47891165519216256)

COMET INFO: train/accuracy [360] : (0.0546875, 1.0)

COMET INFO: train/loss [360] : (0.007132880389690399, 2.565998077392578)

COMET INFO: Others:

COMET INFO: Name : ResNet18_train_finetuning_ood

COMET INFO: notebook_url : <https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych>

COMET INFO: Parameters:

COMET INFO: batch_size : 256

COMET INFO: checkpoint_dir : ./checkpoints

COMET INFO: checkpoint_name : ResNet_finetuning.pth

COMET INFO: epoch_num : 10

COMET INFO: lr : 0.0001

COMET INFO: name : ResNet18_train_finetuning_ood

COMET INFO: seed : 42

COMET INFO: val_batch_size : 256

COMET INFO: Uploads:

COMET INFO: environment details : 1

COMET INFO: filename : 1

COMET INFO: installed packages : 1

COMET INFO: notebook : 2

COMET INFO: os packages : 1

COMET INFO: source_code : 1

COMET INFO:

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET INFO: Please wait for assets to finish uploading (timeout is 10800 seconds)

COMET INFO: Still uploading 1 file(s), remaining 2.77 MB/4.87 MB

```

In [ ]: config = {
    'seed'           : 42,
    'lr'             : 1e-4,
    'epoch_num'      : 10,
    'batch_size'     : 256,
    'val_batch_size' : 256,
    'name'           : 'ResNet18_lp_ood',
    'checkpoint_dir' : './checkpoints',
    'checkpoint_name': 'ResNet_lp.pth'
}

g = set_global_seed(config['seed'])

train_dataloader = DataLoader(
    train_dataset,
    batch_size     = config['batch_size'],
    shuffle        = True,
    drop_last      = True,
    num_workers    = 0,
    worker_init_fn = seed_worker,
    generator      = g
)

valid_dataloader = DataLoader(
    valid_dataset,
    batch_size     = config['val_batch_size'],
    shuffle        = False,
    num_workers    = 0
)

resnet_model = resnet18(
    weights=ResNet18_Weights.IMAGENET1K_V1
)

for parameter in resnet_model.parameters():
    parameter.requires_grad_(False)

resnet_model.fc = torch.nn.Linear(512, 10)

net = ResNetWrapper(
    resnet_model,
    mean = 4,   # Добавили
    std  = 0.1  # Добавили
)

optimizer = optim.Adam(net.parameters(), lr=config['lr'])

```

```

In [ ]: train_final(config, net, optimizer, loss_fn, train_dataloader, valid_dataloader)

```

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET WARNING: As you are running in a Jupyter environment, you will need to call `experiment.end()` when finished to ensure all metrics and code are logged before exiting.

COMET INFO: Couldn't find a Git repository in '/content' nor in any parent directory. Set `COMET\_GIT\_DIRECTORY` if your Git Repository is elsewhere.

COMET INFO: Experiment is live on comet.com <https://www.comet.com/enteralt/cnn-mm-p-2025/3c447300f5284ec7baa0ade209c4leec>

100%|██████████| 10/10 [03:46<00:00, 22.68s/it]

COMET INFO: -----

COMET INFO: Comet.ml Experiment Summary

COMET INFO: -----

COMET INFO: Data:

COMET INFO: display_summary_level : 1

COMET INFO: name : ResNet18_lp_ood

COMET INFO: url : <https://www.comet.com/enteralt/cnn-mm-p-2025/3c447300f5284ec7baa0ade209c4leec>

COMET INFO: Metrics [count] (min, max):

COMET INFO: eval/accuracy [10] : (0.19745222929936307, 0.784203821656051)

COMET INFO: eval/loss [10] : (0.9375370869970625, 2.2064684894586066)

COMET INFO: train/accuracy [360] : (0.0546875, 0.828125)

COMET INFO: train/loss [360] : (0.9286516308784485, 2.565998077392578)

COMET INFO: Others:

COMET INFO: Name : ResNet18_lp_ood

COMET INFO: notebook_url : <https://colab.research.google.com/notebook#fileId=1sBX0xV-QybvDwLmTeo2jM-Ixd75s-Ych>

COMET INFO: Parameters:

COMET INFO: batch_size : 256

COMET INFO: checkpoint_dir : ./checkpoints

COMET INFO: checkpoint_name : ResNet_lp.pth

COMET INFO: epoch_num : 10

COMET INFO: lr : 0.0001

COMET INFO: name : ResNet18_lp_ood

COMET INFO: seed : 42

COMET INFO: val_batch_size : 256

COMET INFO: Uploads:

COMET INFO: environment details : 1

COMET INFO: filename : 1

COMET INFO: installed packages : 1

COMET INFO: notebook : 2

COMET INFO: os packages : 1

COMET INFO: source_code : 1

COMET INFO:

COMET WARNING: To get all data logged automatically, import comet_ml before the following modules: sklearn, torch.

COMET INFO: Please wait for assets to finish uploading (timeout is 10800 seconds)

COMET INFO: Still uploading 1 file(s), remaining 3.58 MB/4.87 MB

При finetuning модель лучше адаптируется к данным вне обучаемого диапазона, так как мы изменяем веса модели. В случае linear probing признаки, которые получает обучаемая голова модели, могут быть испорченные (так как вход из другого распределения). В таблице ниже мы сравнили ассигуру на валидации для всех экспериментов.

	Linear probing	Finetuning
In domain	0.93	0.97
Out domain	0.78	0.93

Конфиги

Существуют аккуратные способы управления конфигами, например использование `ml_collection.Config_Dict` или же библиотеки `Hydra`. Ниже разберем пример на основе `ml_collection` конфига из известной статьи.

```
In [ ]: !pip install ml_collections -q
```

76.7/76.7 kB 4.5 MB/s eta 0:00:00

```
In [ ]: import ml_collections
```

```
In [ ]: # пример взят https://github.com/yang-song/score\_sde\_pytorch/blob/cb1f359f4a

def get_default_configs():
    config = ml_collections.ConfigDict()
    # training
    config.training = training = ml_collections.ConfigDict()
    config.training.batch_size = 128
    training.n_iters = 1300001
    training.snapshot_freq = 50000
    training.log_freq = 50
    training.eval_freq = 100
    ## store additional checkpoints for preemption in cloud computing enviro
    training.snapshot_freq_for_preemption = 10000
    ## produce samples at each snapshot.
    training.snapshot_sampling = True
    training.likelihood_weighting = False
    training.continuous = True
    training.reduce_mean = False

    # evaluation
    config.eval = evaluate = ml_collections.ConfigDict()
    evaluate.begin_ckpt = 9
    evaluate.end_ckpt = 26
    evaluate.batch_size = 1024
    evaluate.enable_sampling = False
    evaluate.num_samples = 50000
    evaluate.enable_loss = True
    evaluate.enable_bpd = False
    evaluate.bpd_dataset = 'test'
```

```

# data
config.data = data = ml_collections.ConfigDict()
data.dataset = 'CIFAR10'
data.image_size = 32
data.random_flip = True
data.centered = False
data.uniform_dequantization = False
data.num_channels = 3

# model
config.model = model = ml_collections.ConfigDict()
model.sigma_min = 0.01
model.sigma_max = 50
model.num_scales = 1000
model.beta_min = 0.1
model.beta_max = 20.
model.dropout = 0.1
model.embedding_type = 'fourier'

# optimization
config.optim = optim = ml_collections.ConfigDict()
optim.weight_decay = 0
optim.optimizer = 'Adam'
optim.lr = 2e-4
optim.betal = 0.9
optim.eps = 1e-8
optim.warmup = 5000
optim.grad_clip = 1.

config.seed = 42
config.device = torch.device('cuda:0') if torch.cuda.is_available() else

return config

get_default_configs()

```

```

Out[ ]: data:
  centered: false
  dataset: CIFAR10
  image_size: 32
  num_channels: 3
  random_flip: true
  uniform_dequantization: false
device: !!python/object/apply:torch.device
- cuda
- 0
eval:
  batch_size: 1024
  begin_ckpt: 9
  bpd_dataset: test
  enable_bpd: false
  enable_loss: true
  enable_sampling: false
  end_ckpt: 26
  num_samples: 50000
model:
  beta_max: 20.0
  beta_min: 0.1
  dropout: 0.1
  embedding_type: fourier
  num_scales: 1000
  sigma_max: 50
  sigma_min: 0.01
optim:
  betal: 0.9
  eps: 1.0e-08
  grad_clip: 1.0
  lr: 0.0002
  optimizer: Adam
  warmup: 5000
  weight_decay: 0
seed: 42
training:
  batch_size: 128
  continuous: true
  eval_freq: 100
  likelihood_weighting: false
  log_freq: 50
  n_iters: 1300001
  reduce_mean: false
  snapshot_freq: 50000
  snapshot_freq_for_preemption: 10000
  snapshot_sampling: true

```

```

In [17]: from hydra import compose, initialize
from omegaconf import OmegaConf

```

```

In [19]: # Пример взят по ссылке
# https://github.com/CompVis/latent-diffusion/blob/a506df5756472e2ebaf9078a1
!cat ./hydra_example.yaml

```



```

model:
  base_learning_rate: 4.5e-6
  target: ldm.models.autoencoder.AutoencoderKL
  params:
    monitor: "val/rec_loss"
    embed_dim: 4
    lossconfig:
      target: ldm.modules.losses.LPIPSWithDiscriminator
      params:
        disc_start: 50001
        kl_weight: 0.000001
        disc_weight: 0.5

    ddconfig:
      double_z: True
      z_channels: 4
      resolution: 256
      in_channels: 3
      out_ch: 3
      ch: 128
      ch_mult: [ 1,2,4,4 ] # num_down = len(ch_mult)-1
      num_res_blocks: 2
      attn_resolutions: [ ]
      dropout: 0.0

data:
  target: main.DataModuleFromConfig
  params:
    batch_size: 12
    wrap: True
    train:
      target: ldm.data.imagenet.ImageNetSRTrain
      params:
        size: 256
        degradation: pil_nearest
    validation:
      target: ldm.data.imagenet.ImageNetSRValidation
      params:
        size: 256
        degradation: pil_nearest

lightning:
  callbacks:
    image_logger:
      target: main.ImageLogger
      params:
        batch_frequency: 1000
        max_images: 8
        increase_log_steps: True

trainer:
  benchmark: True
  accumulate_grad_batches: 2

```

```

In [20]: with initialize(config_path='./', version_base=None):
          config = compose(config_name="hydra_example.yaml")

```

```
config
```

```
Out[20]: {'model': {'base_learning_rate': 4.5e-06, 'target': 'ldm.models.autoencoder.AutoencoderKL', 'params': {'monitor': 'val/rec_loss', 'embed_dim': 4, 'lossconfig': {'target': 'ldm.modules.losses.LPIPSWithDiscriminator', 'params': {'disc_start': 50001, 'kl_weight': 1e-06, 'disc_weight': 0.5}}, 'ddconfig': {'double_z': True, 'z_channels': 4, 'resolution': 256, 'in_channels': 3, 'out_ch': 3, 'ch': 128, 'ch_mult': [1, 2, 4, 4], 'num_res_blocks': 2, 'attn_resolutions': [], 'dropout': 0.0}}, 'data': {'target': 'main.DataModuleFromConfig', 'params': {'batch_size': 12, 'wrap': True, 'train': {'target': 'ldm.data.imagenet.ImageNetSRTrain', 'params': {'size': 256, 'degradation': 'pil_nearest'}}, 'validation': {'target': 'ldm.data.imagenet.ImageNetSRValidation', 'params': {'size': 256, 'degradation': 'pil_nearest'}}}}, 'lightning': {'callbacks': {'image_logger': {'target': 'main.ImageLogger', 'params': {'batch_frequency': 1000, 'max_images': 8, 'increase_log_steps': True}}}, 'trainer': {'benchmark': True, 'accumulate_grad_batches': 2}}
```

```
In [ ]:
```